

Concurrency and Go Routines

Concurrency enabling app to efficiently handle multiple task simultaneously.

Go routines are lightweight threads of execution that make it effortless to write concurrent code, while channels provide a safe way for these goroutines to communicate and coordinate.

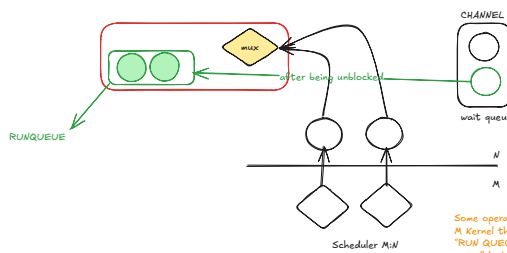
Creating goroutine:

```
go myFunction(); // start a new goroutine
```

Goroutines are managed by the Go runtime scheduler, which efficiently schedules and multiplexes them onto a small number of OS threads.

OS doesn't know anything about the goroutine. Every goroutine has a state to help Scheduler know which goroutine to run based on goroutine state.

- ◊ Running - currently running on kernel thread
- ◊ Runnable - goroutine waiting for kernel thread to run
- ◊ Blocked - goroutines waiting for some conditions (eg. blocked on a channel, syscall, mutex, etc...)



Blocked goroutine should not block the underlying kernel thread!
Each Channel has a recvg (waitq) that is used to store the blocked goroutines which are trying to read data from the channel.

Sendq(waitq) stores blocked goroutines that are trying to send data to the channel.

Unblocked goroutine after channel operation is put into Run queue by the channel itself.

Some operation puts a new goroutine into the run queue.
M Kernel threads access the scheduler to get goroutine to run from "RUN QUEUE". Multiple threads are attempting to access the same area of memory, we will lock this structure with MUTEX from Memory Access Sync.

Channels are a fundamental tool in Go's concurrency toolkit. They enable safe communication and synchronization between goroutines, making it easier to write concurrent programs that are both correct and efficient. When used effectively, channels can help you manage the complexities of concurrent programming in Go and ensure your code behaves predictably in a concurrent environment.

```
ch := make(chan int) // Create an integer channel
ch <- 42 // Send the integer 42 into the channel
data := <-ch // Receive data from the channel (blocks until data is available)
```

Channels can be closed to signal that no more data will be sent. Receivers can use the second return value when receiving data to check if the channel has been closed,

```
close(ch) // Close the channel
data, ok := <-ch
if !ok {
    // The channel has been closed
}
```

Channels are fundamental tool in Go's concurrency toolkit. They enable safe communication and sync between goroutines, making it easier to write concurrent programs that are both correct and efficient. When used effectively, channels can help you manage the complexities of concurrent programming and ensure your code behaves predictably in concurrent environment.

```
select {
case data := <-ch1:
    // Handle data from ch1
case ch2 <- 42:
    // Send data into ch2
case <-time.After(time.Second):
    // Timeout after one second
default:
    // If no other case is ready, do this
}
```

select statement is used to work with multiple channels concurrently. It allows you to choose which channel to send or receive data from when multiple channels are involved. This is useful when you need to handle various asynchronous events or communication patterns.

Prevention of data races. Data races occur when multiple goroutines access and modify shared data without proper synchronization. Goroutines and channels promote safe concurrent programming. Since goroutines execute concurrently but not necessarily simultaneously, you can avoid race conditions by ensuring that only one goroutine can access a piece of shared data at a time through proper synchronization with channels or other synchronization primitives.

```
var wg sync.WaitGroup

func main() {
    for i := 0; i < 5; i++ {
        wg.Add(1) // Increment the WaitGroup counter
        go worker(i)
    }

    wg.Wait() // Wait for all workers to finish
}

func worker(id int) {
    defer wg.Done() // Decrement the WaitGroup counter when done
    // Worker's work
}
```

sync.WaitGroup is a synchronization primitive in Go used to wait for a collection of goroutines to finish before proceeding. Useful when you want to wait for a specific number of goroutines to complete their work.